Algorithmic Trading, and How We Verify It

Giolit Trading Risk Verifier 1.0.0 — Tutorial 3

Contents

1. What algorithmic trading is . . . 1
2. How a market works: the order book . . . 2
3. Orders and their life cycle . . . 2
4. The components of a trading system . . . 2
5. Strategy families . . . 3
6. Protocols . . . 4
7. Standards and regulation . . . 4
8. How trading systems fail . . . 5
9. How the verifier models a trading system . . . 6
10. What we verify . . . 7
11. How results are reported . . . 8
12. From regulation to constraints . . . 8
13. Glossary . . . 9

Tutorial 3 of 3 for users of the Giolit Trading Risk Verifier 1.0: the domain, its standards and protocols, and how the verifier models a trading system, what it verifies and how it reports. No background in formal methods is assumed.

---

1. What algorithmic trading is

Algorithmic trading is trading in which a computer program decides whether, when, how much and at what price to buy or sell, and sends the orders itself, with little or no human intervention per order. It covers:

- **Execution algorithms**: a human or a portfolio system decides *what* to trade ("buy 2 million shares of INFY today"); the algorithm decides *how* — slicing the parent order into child orders over time, across venues, along the traded volume, to reduce market impact and cost.
- **Trading strategies**: the program also decides *what* to trade — market making, arbitrage, statistical and trend-following strategies.
- **High-frequency trading (HFT)**: strategies whose edge depends on reacting within microseconds to milliseconds, with high message rates and short holding periods.

Algorithms trade a large share of the volume on most electronic equity, futures, options and FX markets. Because they act fast and without a human in the loop, an error in their logic repeats at machine speed.

That is why regulators require firms to test algorithms, to put risk controls in front of them, and to be able to stop them.

2. How a market works: the order book

An exchange (a **venue**) keeps, for every instrument, a **limit order book**: the buy orders (**bids**) and the sell orders (**asks** or offers) that are waiting to trade, each with a price and a quantity.

```
        ASK (sell orders)            price      qty
                                     100.15     800
                                     100.10     300   <- best ask
        ------------------------------------------ spread = 0.05
                                     100.05     500   <- best bid
                                     100.00    1200
        BID (buy orders)
```

- The **best bid** and **best ask** are the highest buy and lowest sell price; their difference is the **spread**.
- Prices move in steps of the **tick size** (0.05 INR here); quantities often in **lots**.
- An incoming order that can trade against the other side **matches**: most venues use **price–time priority** (better price first, then earlier order first). What does not match rests in the book.
- A **fill** (execution) can be **partial**: an order of 500 may fill 300 now and 200 later, or never.
- Venues add **price bands** and **circuit breakers** (orders far from the reference price are rejected; trading halts after large moves), and **auctions** at the open and close.

Market data is the stream of book updates and trades the venue publishes. A strategy sees the market only through this stream, and it is always slightly out of date.

3. Orders and their life cycle

Order types. *Limit* (at this price or better), *market* (at any price), and variants: *IOC* (immediate or cancel), *FOK* (fill or kill), *stop*, *iceberg* (shows part of its size), *post-only* (never takes liquidity). **Time in force**: day, GTC (good till cancelled), GTD.

Life cycle. In FIX terms (and the verifier's model):

```
NewOrder         Ack                PartialFill*        Fill
————————> PendingNew ——> New ————————————————> PartFilled ——> Filled
              | Reject      | CancelRequest       | CancelRequest
              v             v                     v
          Rejected     PendingCancel —— CancelAck ——> Cancelled
                            |   (fills can still arrive before the CancelAck)
                            └—— CancelReject ——> back
```

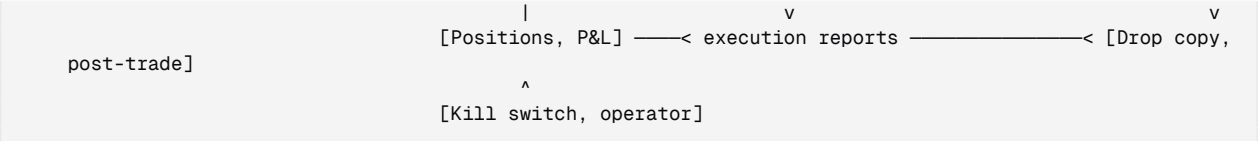
Every arrow is a **message**, and messages take time. Between the moment a strategy sends an order and the moment it learns the result, the market moves, other orders fill, and other messages arrive. A strategy's picture of its own orders is therefore always a little behind the venue's — the source of most of the failures in section 8:

- an order the strategy believes is cancelled can still fill (a **late fill**);
- a cancel can be rejected because the order has already filled;
- a fill can be reported as two messages (the trade and the order's new status), in either order;
- the venue can cancel or expire an order on its own.

Position and P&L. The **position** is shares bought minus shares sold; **open quantity** is what is still working in the book; **cash** is money received minus money paid. Mark-to-market P&L values the position at the current price.

4. The components of a trading system

```
   market data ——> [Feed handler] ——> [Strategy / signal] ——> [OMS / EMS] ——> [Pre-trade risk] ——> [Gateway
     ] ——> Venue
                                          ^                       |                                    |
```



Component	What it does
Feed handler	decodes the venue's market data and builds the book
Strategy / signal	decides what to trade: quotes, spreads, signals (a model's output), schedules
OMS / EMS	order and execution management: parent and child orders, their states, amendments
Smart order router (SOR)	chooses the venue(s) for each child order
Pre-trade risk gateway	checks every order before it leaves: size, value, price band, position and exposure limits, message rates, kill switch
Gateway / FIX engine	speaks the venue's protocol, keeps the session
Position keeping	books fills into positions and cash
Kill switch	stops all new orders and cancels working ones, on an operator's or a control's command
Drop copy	a copy of every execution report, sent by the venue or broker to a separate system: the independent record used for surveillance and reconciliation
Post-trade	clearing, settlement, reconciliation, reporting

The verifier looks at the parts that decide and control orders — the strategy, its own bookkeeping and the risk controls — together with a model of the venue's protocol and of the market.

5. Strategy families

Family	Idea	What typically goes wrong
Market making	quote a bid and an ask, earn the spread, keep inventory small	inventory beyond limits, quotes that cross the market, two quotes per side during cancel/replace
TWAP / VWAP / POV	work a parent order evenly over time / along the volume curve / at a share of the volume	overfill (child orders exceed the parent), a slice that turns a sell into a buy, a schedule that can never complete
Implementation shortfall	balance market impact against price risk	the same, plus cost
Pairs / statistical arbitrage	trade two related instruments against each other	**legging risk**: one leg fills, the other is rejected or does not fill
Cross-venue and spot–futures arbitrage	buy where cheap, sell where dear	both venues fill the same quantity (overfill), positions out of balance after partial fills
Hedging (delta, beta)	keep the exposure of a book neutral	over-hedging, a hedge sent before the last one is done

Family	Idea	What typically goes wrong
Momentum, mean reversion, trend	trade on signals	position limits, order rates, behaviour after a kill switch

6. Protocols

FIX (Financial Information eXchange) is the industry's common language for orders and executions, maintained by the FIX Trading Community. A FIX message is a list of `tag=value` fields:

```
8=FIX.4.4|35=D|11=O1|55=INFY|54=1|38=100|40=2|44=1500.05|60=20261009-09:15:00.000|
```

Tag	Name	Here
35	MsgType	`D` new order single (`F` cancel request, `8` execution report, `9` cancel reject, `W`/`X` market data)
11 / 41	ClOrdID / OrigClOrdID	the order's id / the order a cancel refers to
55	Symbol	INFY
54	Side	1 buy, 2 sell
38 / 44	OrderQty / Price	100 at 1500.05
40	OrdType	1 market, 2 limit
150 / 39	ExecType / OrdStatus	in execution reports: 0 new, F trade, 4 cancelled, 8 rejected
32 / 31	LastQty / LastPx	the quantity and price of a fill

Versions in use: **FIX 4.2** and **4.4** (most widely deployed), **FIX 5.0 SP2** over the **FIXT 1.1** session layer. Related standards from the same community:

- **FIX Orchestra**: a machine-readable description of a venue's *rules of engagement* — its messages and its order state machine. The verifier can read the state machine from it.
- **FIXatdl**: a description of an algorithm's parameters (types, ranges) as an OMS shows them to a trader. The verifier can verify a strategy for every parameter value it allows.
- **SBE** (Simple Binary Encoding) and **FAST**: binary encodings for low latency, mostly for market data.

Venue-native protocols. Many venues also (or only) offer their own binary protocols, for example NASDAQ ITCH (market data) and OUCH (orders), CME iLink and MDP, Eurex and BSE ETI, the NSE's native trading and tick-by-tick feeds. **Broker APIs** (for example CTP for Chinese futures) often report a fill as two callbacks — the trade and the order's new status — which the verifier models with the protocol option `fill_reports`.

The verifier's protocol model is the FIX 4.4 order life cycle (section 3). Venue differences are options: `late_fills`, `exchange_cancels`, `fill_reports`, or a FIX Orchestra file.

7. Standards and regulation

Rules differ by jurisdiction and change; this section is an orientation, not legal advice. Check the current text with your compliance team.

Common themes. Regulators and exchanges in most markets require a firm that trades algorithmically to:

1. **test** its algorithms before deployment and after every material change, including in stressed conditions;

2. run **pre-trade risk controls** on every order: maximum order size and value, price collars, position and credit limits, message rate limits;
3. have a **kill switch** that stops new orders and cancels working ones immediately;
4. **monitor** trading in real time and keep records for surveillance and audit;
5. keep **governance**: approval of algorithms, change management, an annual review.

Examples by region.

Region	Examples
India	SEBI's circulars on algorithmic trading for stock brokers and exchanges (risk controls, approval of algorithms by the exchange, testing, order-to-trade ratio limits), and its framework for retail participation in algorithmic trading; the exchanges' (NSE, BSE, MCX) technical and testing requirements
European Union / UK	MiFID II Article 17 and its RTS 6 (Commission Delegated Regulation (EU) 2017/589): testing of algorithms, pre-trade controls (price collars, maximum order value and volume, maximum messages), kill functionality, real-time monitoring, an annual self-assessment; the FCA's equivalent rules in the UK
United States	SEC Rule 15c3-5 (the Market Access Rule): pre-trade risk controls for broker-dealers with market access; FINRA guidance on the supervision of algorithmic trading; the CFTC's electronic trading risk principles for futures
Industry	FIA guidance on automated trading risk controls; IOSCO principles

Reference data standards used in trading systems: ISO 10383 (market identifier codes, MIC), ISO 4217 (currencies), ISO 6166 (ISIN), ISO 20022 (post-trade messages).

Where verification fits. Testing requirements ask for evidence that algorithms and controls behave as intended under the conditions that can occur. Tests and simulations sample those conditions; a proof covers all of them under stated assumptions. The verifier's certificates are evidence for that part of the obligation — they do not replace your governance, your approvals or your monitoring.

8. How trading systems fail

Most expensive failures are not wrong formulas. They are **interleavings**: messages that arrive in an order the programmer did not consider, and state that is briefly out of date.

Failure	How it happens
Two orders through one limit	a check looks at the position but not at the orders still in flight; two orders pass before either fills
Overfill	a router sends the same quantity to two venues and both fill; an execution algorithm re-sends a slice that was still working
Late fill	an order fills after the strategy treated its cancel as done, and the strategy has already replaced it
Partial fills	code written for "filled or not" treats a partial fill as complete and sends the next clip

Failure	How it happens
Separate reports	the trade and the order's status arrive as two messages; between them, the strategy's counters disagree
Legging	one leg of a pair fills, the other is rejected; the next signal opens another pair
Kill switch gaps	a fill handler sends a new order after the kill switch, or the kill switch reaches only one venue
Stale risk state	the risk check runs on another thread and reads a position that is not yet updated
Numeric edge cases	a floating-point rounding, an unsigned subtraction that wraps around, a value that becomes NaN
Deployment	old code reactivated by a configuration flag on some servers but not others

The best-known example of the last kind is the August 2012 incident at Knight Capital, where a deployment error reactivated obsolete order-routing logic and produced a loss of about USD 440 million in under an hour. The others are the everyday kind: each is found in seconds by the verifier in the package examples, and several were found in unmodified open-source strategies.

None of these needs an unusual market — only an unusual *order of messages*, which a back-test on historical data almost never contains, because the historical record has one order of messages per day.

9. How the verifier models a trading system

A transition system. The verifier turns your specification into a precise mathematical model: a **state** and the **events** that change it.

State	
Market	best bid, ask, last price, volume (per venue and symbol)
Orders	each order's side, quantity, limit, filled quantity and life-cycle state
Account	position, bought, sold, cash (per symbol), exposures
Strategy	its parameters and its own state variables (from GTSL, or from your code)
Controls	the kill switch; for asynchronous controls, the last published position
Time	the clock (and the session)

Event	Source
`market_update`	the market: any new bid, ask, last price within the ranges and assumptions
`ack`, `reject`, `fill` (partial or complete), `cancel_ack`, `cancel_reject`, `exchange_cancel`, `fill_report`	the venue, following the protocol
`clock`, `session_close`	time
`kill_switch`	the operator

One step is one event. After an event, the strategy's handlers for that event run; every order they send passes the risk controls; the assumptions must still hold; then every constraint is checked. The verifier considers **every** sequence of events and **every** value the market and the venue can choose — not a sample.

Nondeterminism is the adversary. Wherever the real world could do several things (the next price, which order fills, how much, whether a cancel arrives first), the model allows all of them. A proof therefore covers every possibility; a violation is one concrete choice of them.

Two models of the orders.

- The **slot model** tracks up to K live orders (default 4) with their exact life cycle. It is exact: every violation it finds is a real run.
- The **aggregate model** tracks the total open quantity per side, the number of live orders, and one *watched* order. It over-approximates any number of orders, so its proofs hold **for any number of orders**.

They are verified side by side; each answers what it can answer soundly.

Exact arithmetic. Prices are counted in ticks, money in money units, quantities in shares: integers, with no rounding. Strategies that compute with `double` are modelled as IEEE-754 binary64, bit for bit as C/C++ compute it.

Your code as the model. With the importer, the strategy part of the model is built from your C++, Python or Java source: its handlers, state, conditions and orders. A binding states what the code's events and fields mean; nothing is approximated without being reported.

The engines. Several independent engines work on the model at once and the first decisive answer wins:

- **proof engines** search for an *inductive invariant*: a statement about the state that holds at the start, that every event preserves, and that rules out every violation. Finding one proves the constraint for all runs;
- **search engines** look for violating runs, with random exploration and a search guided towards the violation.

All engines are Giolit's own; no third-party solver is involved in a verdict.

10. What we verify

Question	Constraint form	Example
Can a limit ever be broken?	`always COND`	`always position <= max_long`
Is every order acceptable when it is sent?	`always on new_order: COND`	`order.qty <= clip`, `notional(order) <= 500000 INR`, never crossing the market
Does the kill switch work?	`after kill_switch: never new_order`	
Is an order final once cancelled?	`for each order o: after cancelled(o): never fill(o)`	(holds only if the venue has no late fills)
Does the strategy's bookkeeping match the venue's?	`always COND` over its own variables	`always filled == bought`
Can it do its job?	`feasible COND`, `feasible at end: COND`	`feasible at end: bought == quantity`
Which values can it reach?	headroom, optimisation	worst position, worst implementation shortfall
Do the controls protect against any algorithm?	`mode controls_only;`	
Does production behave like the model?	conformance replay of FIX logs	

Automatic **sanity checks** run with every verification: the assumptions allow trading at all, every rule can fire, every action can execute, every risk control can reject an order.

11. How results are reported

Output	Content
Verdict per constraint	PROVEN, PROVEN* (for at most K live orders), VIOLATED, FEASIBLE, INFEASIBLE, UNDECIDED — with the method and the time
Violating run	event by event, with the strategy's orders and the state after each event; shortened to the events that matter; also as a **FIX 4.4 log**, JSON and a CSV market path
Proof	the inductive invariant, written as relations over the trading state (`open_qty(buy) + position <= 1000`)
Headroom	per proven limit: the worst value any run reaches, the unused share, the Giolit Risk Resilience Score (1–5 stars)
Certificate bundle	a PDF certificate (or violation/verification report), a statement signed by the licensee with the SHA-256 of the specification and of every evidence file, and `verify_certificate.py` to check it offline
CI formats	JSON, JUnit XML, comparison with a baseline (regressions)
Conformance report	every log message the specification excludes, every order the design would not send

A PROVEN verdict always states its scope: the specification's assumptions, ranges, protocol options and order model. A VIOLATED verdict is always a concrete run of the exact model that you can replay.

12. From regulation to constraints

Requirement (typical wording)	In the verifier
Maximum order size / value	`always on new_order: order.qty <= max_qty`, `notional(order) <= max_value` — controls-only proof
Price collars	`always on new_order where order.side == buy: order.price <= market.ask + band`
Position and exposure limits	`always position <= max_long`, `always gross_exposure <= limit`, per symbol and venue
Kill functionality	`after kill_switch: never new_order`; `on kill_switch { cancel_all(); }` and its effect
Algorithms tested against disorderly conditions	every market move within the ranges, every order of messages, late fills, rejects, venue cancels
Controls independent of the algorithm	`mode controls_only;`: proved for any strategy
Records and evidence	the signed certificate bundle, archived with each release
Change management	the CI gate with a baseline: a lost proof is a regression

Requirement (typical wording)	In the verifier
Real-time monitoring	conformance replay of the drop copy, `--follow`

13. Glossary

Term	Meaning
Ack	the venue's acknowledgement that an order is live
Aggregate model	the model of any number of orders by their totals; its proofs cover any number of orders
Assumption	a fact about the world the verification may rely on (stated in the specification)
Binding	the file that says what your code's handlers, fields and calls mean
Constraint	a rule the strategy must never break
Counterexample, run	a concrete sequence of events that breaks a constraint
Drop copy	an independent copy of the execution reports
Fill, partial fill	an execution of all or part of an order
GTSL	the Giolit Trading Specification Language
Inductive invariant	a statement that holds initially and after every event, and implies the constraint
Leaves	the quantity of an order still open
Legging risk	one leg of a multi-leg trade executes and another does not
Open quantity	the leaves of all live orders of a side
Position	shares bought minus shares sold
Slot model	the exact model of up to K live orders; its runs are real
Tick	the minimum price step

Giolit Labs, Thrissur, Kerala, India — www.giolit.com — business.giolitlabs@gmail.com